Unit – 3 (continued)

Software Architecture & Hypermedia Design

Architecture • Genres • Styles • Architectural Design • Interface • Content • Navigation • OOHDM



MCA – 212 • Object-Oriented Software Engineering

Dr Madhavi Dave

Learning Objectives

What students should be able to do after this module

01 • Architecture

- Explain what software architecture is and why it matters.

02 • Genres & Styles

- Classify systems by architectural genre and choose a suitable style.

03 • Architectural Design

- Produce a context diagram and refine it into a layered architecture.

04 • Interface & Content

- Apply the golden rules of interface design and design content objects.

05 • Navigation Design

- Choose linear, hierarchical, network or mixed navigation structures.

06 • OOHDM

- Walk a Web/hypermedia application through all four OOHDM phases.

- A single running case study — an E-Learning Portal — threads through the interface, content, navigation and OOHDM sections.

SECTION 1

Software Architecture

The overall structure that everything else builds on

What Is Software Architecture?

The system's overall structure and the discipline of designing it

- Software architecture is the set of principal design decisions about a system: how it is decomposed into components, how those components interact, and the properties of each.
- It represents the earliest, highest-leverage design decisions — mistakes here are the most expensive to fix later.
- Architecture is documented through multiple views, because no single diagram can capture structure, behavior, deployment and data all at once.

Components

- The computational elements (modules, services, objects) and their responsibilities.

Connectors

- How components communicate — calls, events, shared data, message queues.

Constraints

- Rules the architecture must satisfy — performance, security, budget, platform.

Architectural Views

The view model — one structure, several perspectives

View	Answers	Typical Diagram
Logical	What functionality does the system provide?	Class / object diagrams
Process	How do components run and communicate at runtime?	Sequence / activity diagrams
Development	How is the code organized for the team?	Package / component diagrams
Physical	How is it deployed onto hardware?	Deployment diagrams
Scenarios	Do the other four views actually satisfy the use cases?	Use case diagrams

Quality Attributes & Trade-offs

Architecture is where non-functional requirements are won or lost

Performance

- Response time and throughput under expected load.

Security

- Protecting data and functions from unauthorized access.

Availability

- Staying operational despite failures.

Modifiability

- How cheaply the system can absorb future change.

Scalability

- Handling growth in users, data or transactions.

Usability

- How easily real users accomplish their goals.

No free lunch

Quality attributes routinely conflict — e.g. maximizing security typically costs some usability and performance. Architecture is the art of making these trade-offs explicit and deliberate.

SECTION 2

Architectural Genres

Broad families of application, each with its own recurring structure

What Are Architectural Genres?

The application domain narrows down which architecture fits

- A genre is a broad category of software distinguished by the problem it solves — much as literary genres group books by their kind of story.
- Systems in the same genre tend to reuse the same handful of architectural styles, because they face similar constraints and data patterns.
- Identifying a system's genre early is a fast way to shortlist candidate architectural styles instead of starting from a blank page.

Genre	Description	Example
Data-intensive / DB-centric	Structure revolves around a large shared database	ERP, inventory systems
Client/Server	Work is split between requesting clients and a serving backend	Banking, e-mail systems
Web / Hypermedia	Content-rich, link-navigated applications	E-commerce, e-learning portals
Real-time / Embedded	Strict timing constraints drive every decision	Industrial control, avionics
AI / Rule-based	Behavior driven by inference over a knowledge base	Medical diagnosis, recommender systems

SECTION 3

Architectural Styles

Reusable structural patterns — the vocabulary of architecture

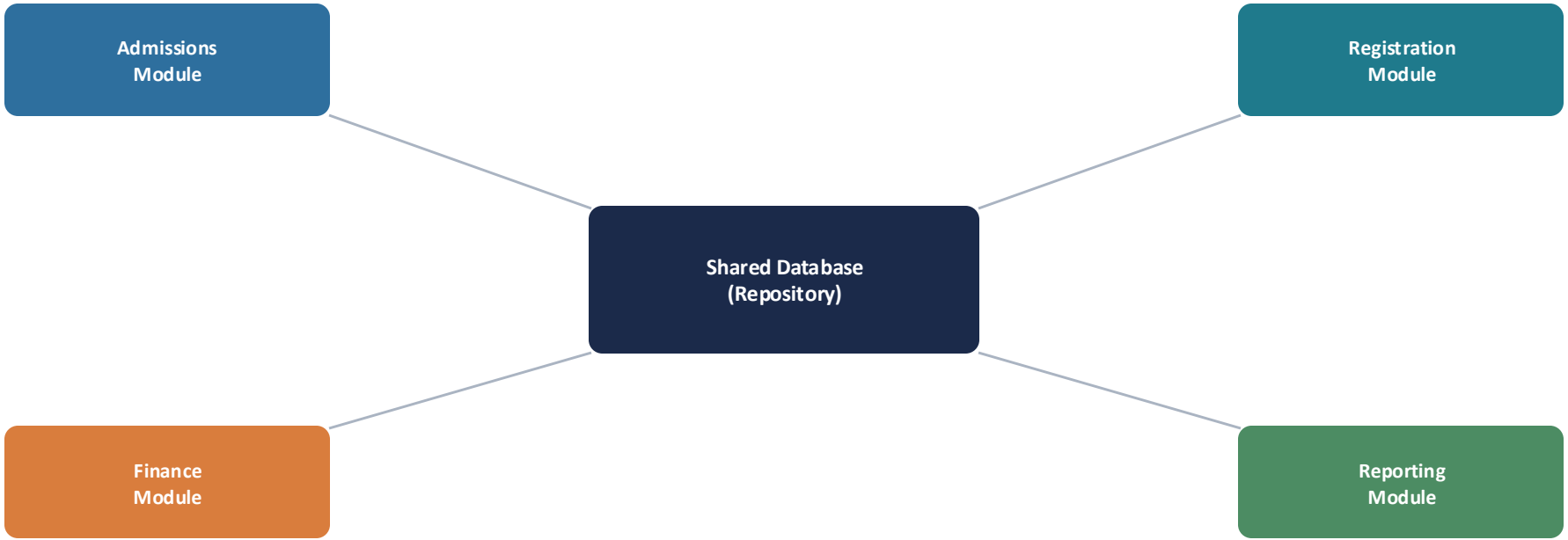
Five Classic Architectural Style Families

Pick the family whose core idea matches your problem

Style	Core Idea	Typical Use
Data-centered (Repository)	A shared data store that independent components read/write	Large databases, compilers
Data-flow (Pipe & Filter)	Data streams through a chain of transforming filters	Compilers, signal/batch processing
Call-and-return (Layered)	Hierarchical control; each layer hides detail from the one above	Business apps, OS, protocol stacks
Object-Oriented	Independent objects collaborate purely via messages	GUI toolkits, simulations
Event-based (Implicit Invocation)	Components broadcast/react to events with no direct calls	GUIs, plugin systems, IoT

Data-Centered (Repository) Style

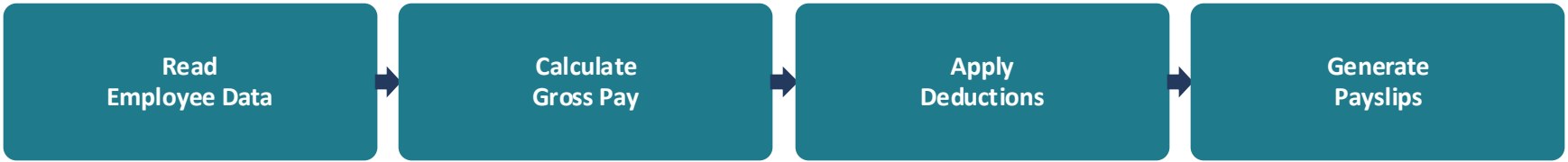
Case study: a University ERP built around one shared database



Components never talk to each other directly — every interaction happens by reading or writing the shared repository.

Data-Flow (Pipe & Filter) Style

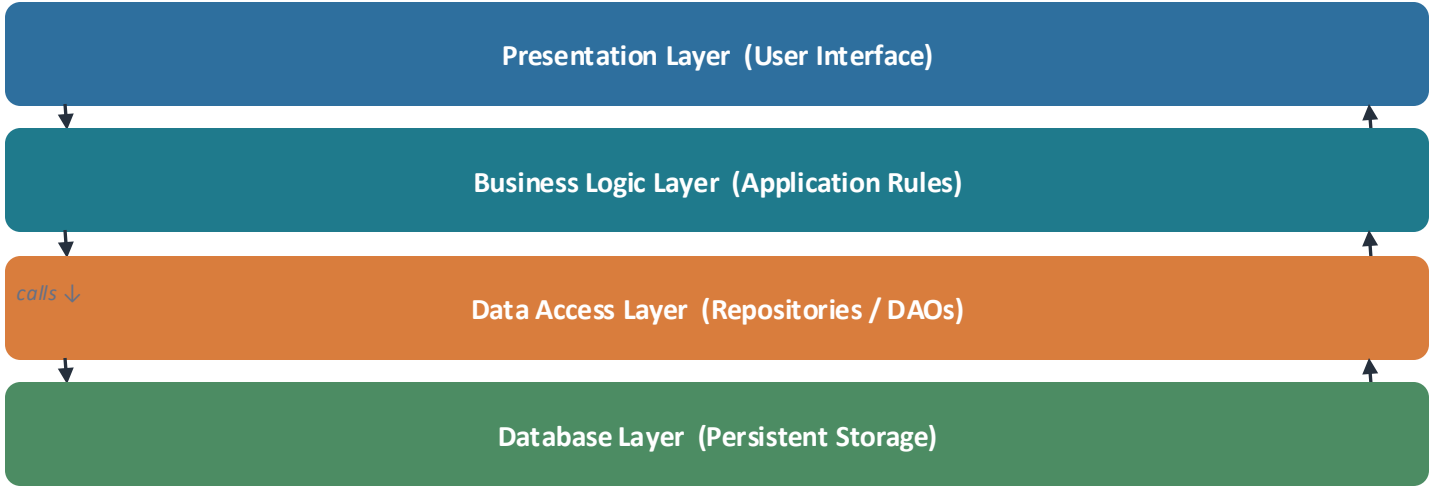
Case study: a payroll batch-processing pipeline



- Each filter reads a stream, transforms it, and passes the result down the pipe — filters know nothing about each other's internals.
- Filters can be reordered, replaced, or run in parallel as long as their input/output stream format stays the same.
- Weakness: not ideal for interactive systems that need immediate two-way conversation with a user.

Call-and-Return: The Layered Style

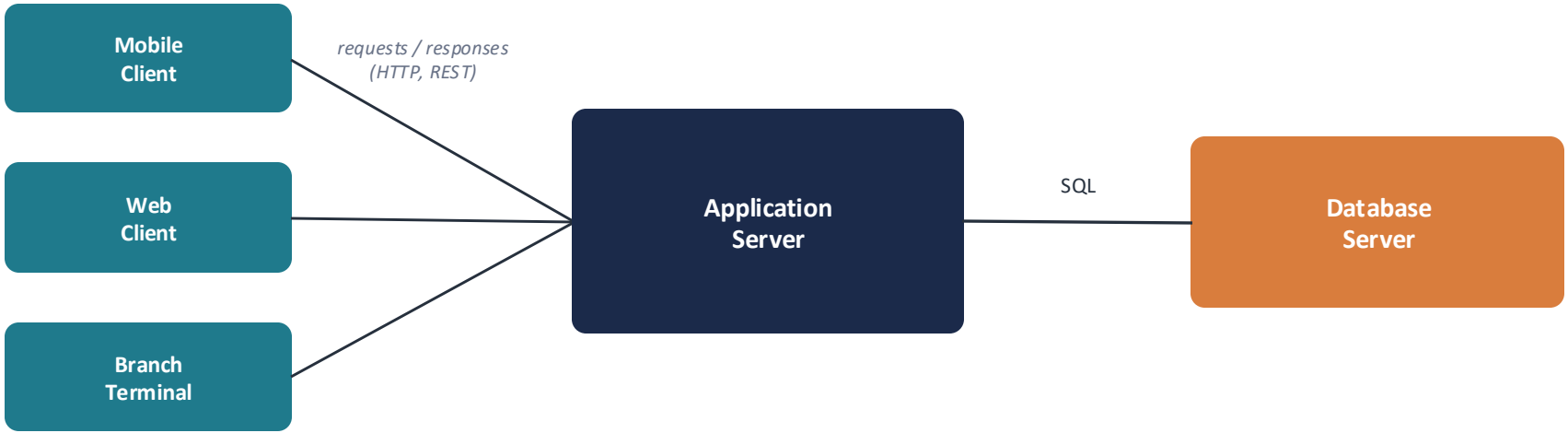
Each layer only calls the layer directly beneath it



A layer may call downward and receive results back upward, but must never skip a layer or call upward — this keeps each layer independently replaceable.

Client-Server Style

Case study: a banking application



Why this fits

Many clients share one server, which centralizes business rules and security; the server itself is decoupled from storage via the database server, so either tier can scale independently.

Choosing a Style

Match the style's strengths to your dominant concern

If your priority is...	Favor this style
A single, consistent source of truth for data	**Data-centered / Repository**
Transforming a stream of records step by step	**Data-flow / Pipe & Filter**
Isolating change so lower levels can evolve safely	**Layered**
Serving many remote users against shared logic	**Client-Server**
Loose coupling between many optional plugins	**Event-based / Implicit Invocation**

In practice

Real systems mix styles: a Web application is often layered internally, client-server at its boundary, and event-based in its UI — pick the dominant style per concern, not one style for the whole system.

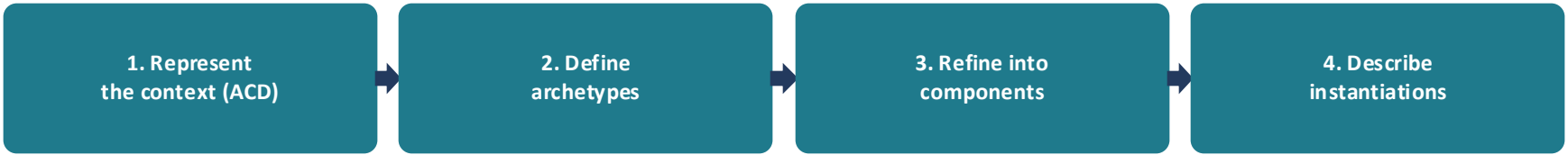
SECTION 4

Architectural Design

Turning requirements into a concrete, structured architecture

The Architectural Design Process

From system context to described components



1 • Context

- Draw an Architectural Context Diagram showing the system and every external entity it exchanges data with.

2 • Archetypes

- Identify the abstract building blocks the system is made of (e.g. Node, Session, Repository).

3 • Components

- Refine archetypes into concrete components and choose a style to connect them.

4 • Instantiate

- Map components onto real classes, modules and deployment nodes.

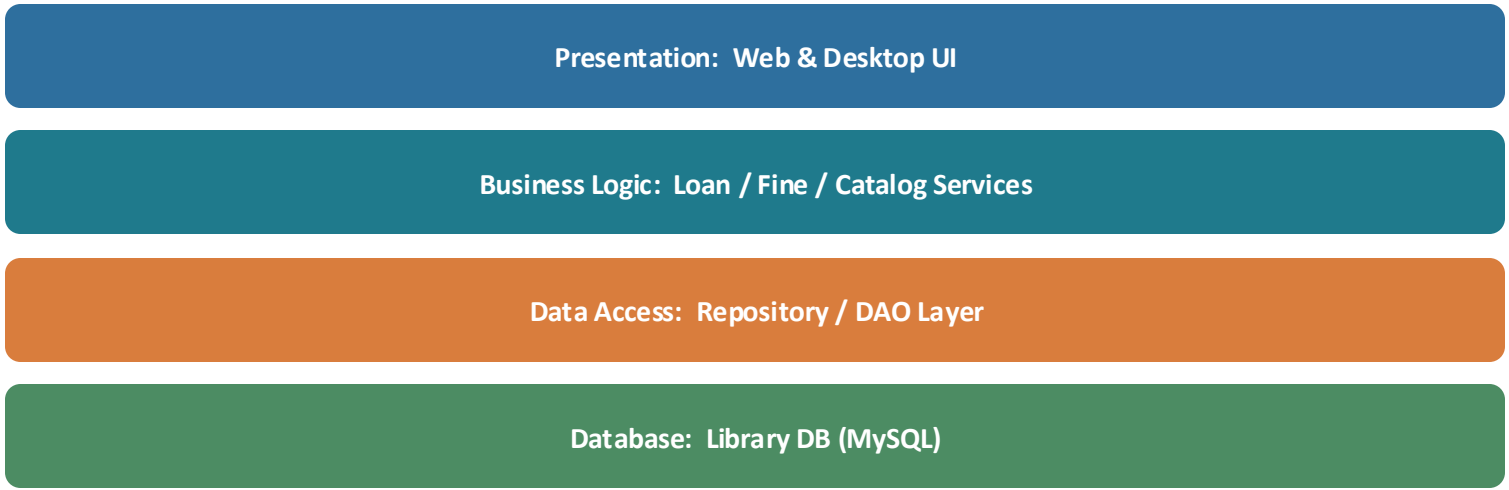
Architectural Context Diagram (ACD)

Case study: Library Management System in its environment



Case Study — Layered Architecture for the Library System

Refining the ACD into a concrete, style-based architecture



Design decision
The Business Logic layer depends only on Data Access interfaces (not concrete classes), so the database technology can be swapped without touching business rules.

SECTION 5

Interface Design

Making the system's power accessible and predictable to people

Golden Rules of Interface Design

Three principles that resolve most interface arguments

1 • Place the User in Control

- Allow interruption and undo.
- Hide technical internals from casual users.
- Let users customize the interaction.

2 • Reduce Memory Load

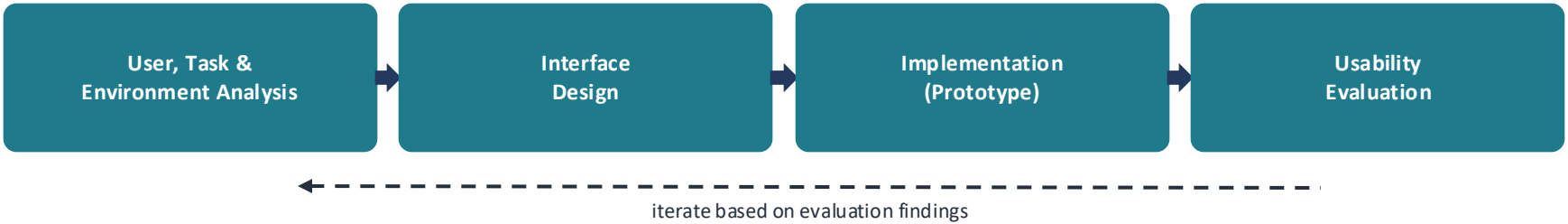
- Establish meaningful, sensible defaults.
- Define shortcuts for frequent actions.
- Disclose information progressively.

3 • Make the Interface Consistent

- Consistent way to present and access information.
- Consistent visual layout across screens.
- Consistent input mechanisms throughout.

Interface Design Process

A short, iterative loop — not a one-shot activity



- Analysis: understand who the users are, what tasks they perform, and the environment they work in.
- Design: sketch screen layouts, navigation and controls before writing production code.
- Implementation: build a working prototype — low-fidelity first, then higher fidelity.
- Evaluation: test with real users; findings feed straight back into another design iteration.

Case Study — E-Learning Portal Interface Elements

Three categories of interface element, applied to one screen

Navigation Elements

- Top menu (Home / Courses / Dashboard).
- Breadcrumb trail on course pages.
- "Back to catalog" link.

Informational Elements

- Course description and syllabus.
- Progress bar showing % complete.
- Instructor rating and review count.

Functional Elements

- "Enroll now" button.
- Search bar with live filtering.
- Video playback controls.

SECTION 6

Content Design

Identifying and organizing everything the application will present

What Is Content Design?

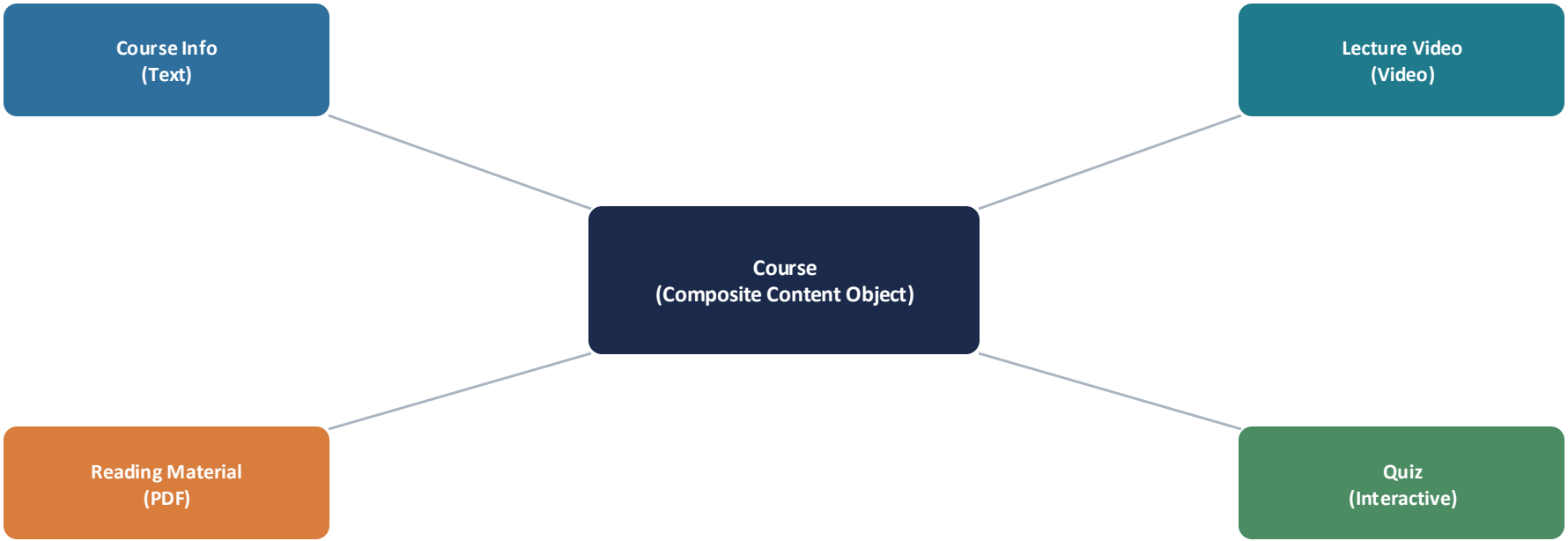
Designing the content, independent of navigation and interface

- Content design identifies, creates and organizes every content object a Web/hypermedia application will present — before deciding how users move between them.
- A content object is any self-contained unit of information: a paragraph, an image, a video clip, a downloadable file, or a composite made of several of these.
- Separating content design from navigation and interface design lets each concern evolve independently — the same content can be re-navigated or re-skinned later.

Content Type	Example (E-Learning Portal)
Text	Course description, lesson transcript
Graphics / Images	Instructor photo, diagram, course thumbnail
Audio / Video	Recorded lecture, narrated slideshow
Interactive / Executable	Quiz, code sandbox, discussion form

Case Study — Content Objects for an Online Course

A composite content object, assembled from simpler ones



Each part is designed once and can be reused inside other composite objects (e.g. the same video in a "Highlights" reel).

SECTION 7

Navigation Design

Defining the paths users may take through the content

Navigation Structures I

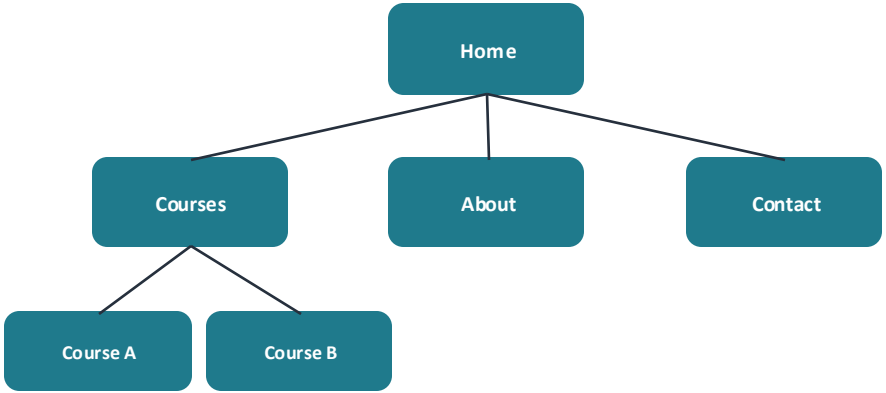
Linear and Hierarchical structures

Linear



Fixed, one-way order — the reader progresses step by step (e.g. a guided walkthrough).

Hierarchical

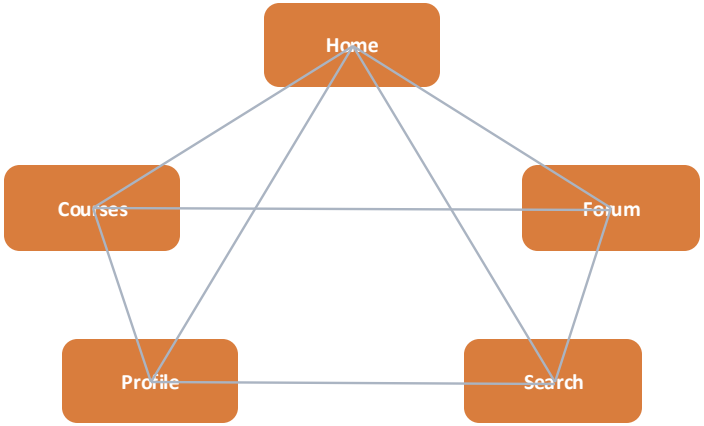


A tree of nodes reached by drilling down from a root (e.g. a course catalog by category).

Navigation Structures II

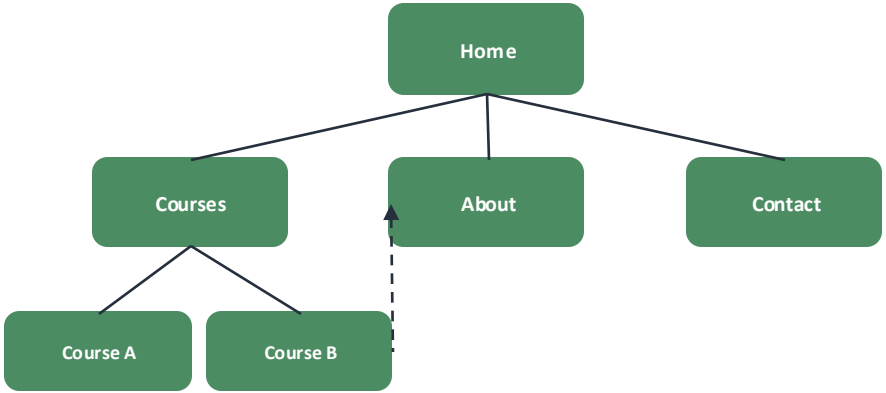
Network and Mixed structures

Network



Every node links to several others — the reader chooses their own path (e.g. a wiki or forum).

Mixed



A hierarchy with a few extra cross-links (shortcuts) layered on top — most real Web apps use this.

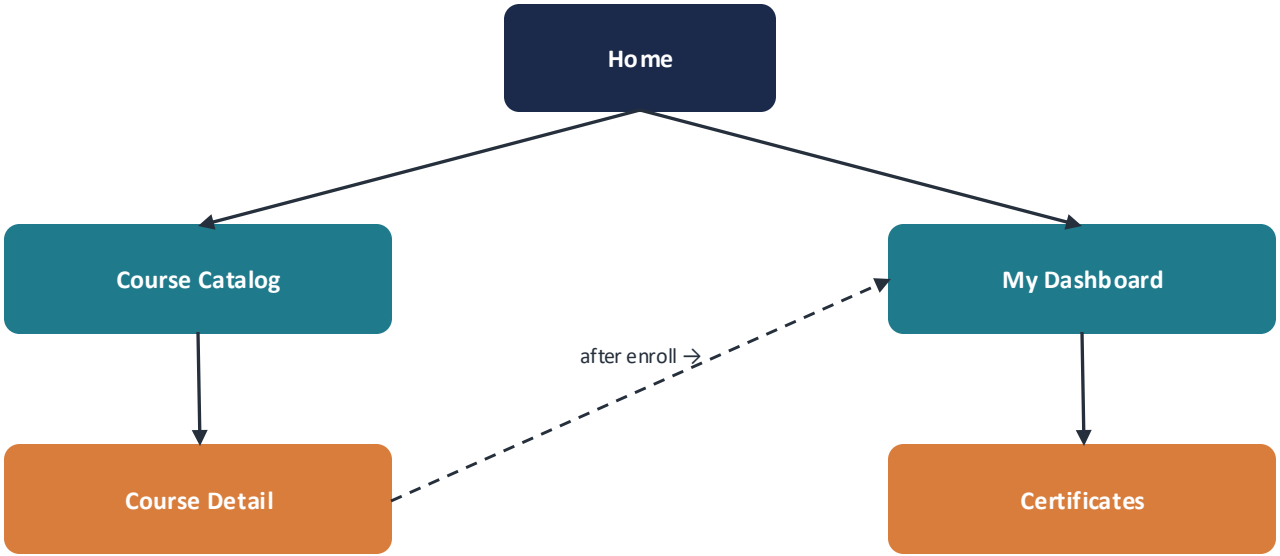
Navigation Design Elements

The recurring building blocks of any navigation design

Element	Purpose
Node	A single navigable unit of content (a page or screen)
Link	A directed connection a user can follow from one node to another
Index	A list of nodes grouped by some criterion, for direct access
Guided Tour	A predefined linear path through a subset of nodes
Menu	A persistent set of links to the most important nodes

Case Study — Navigation Design for the E-Learning Portal

A mixed structure with one deliberate cross-link



The dashed cross-link is a deliberate shortcut: after enrolling from Course Detail, the user is dropped straight onto their Dashboard instead of climbing back up the tree.

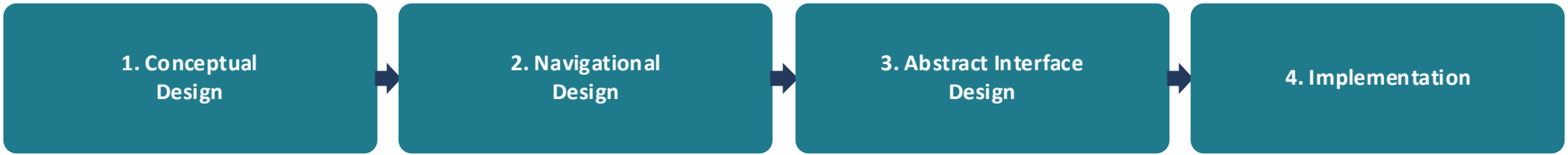
SECTION 8

Object-Oriented Hypermedia Design Method

OOHDM — a four-phase method for designing Web/hypermedia applications

OOHDM — Four Phases

Conceptual model → Navigation → Interface → Implementation



1 • Conceptual

- Build a domain (class) model of the application, exactly like OO analysis.

2 • Navigational

- Define navigation classes — nodes, links, contexts — as views over the conceptual model.

3 • Abstract Interface

- Define Abstract Data Views (ADVs) mapping perceptible objects to interface widgets.

4 • Implementation

- Map the abstract interface objects onto a concrete platform (Web, mobile, desktop).

Case Study — OOHDM Applied to the E-Learning Portal

Walking one application through all four phases

Phase	Deliverable	E-Learning Portal Example
Conceptual	Domain class model	Course, Instructor, Student, Enrollment
Navigational	Nodes, links, navigational contexts	CourseNode, LessonNode; context "Courses by Category"
Abstract Interface	Abstract Data Views (ADVs)	Course list = scrollable ADV; Enroll = activates enroll()
Implementation	Concrete platform mapping	ADVs rendered as HTML/CSS/JS components

How It All Connects

From structural decisions down to the pixels a user touches



- Architecture and style set the system's large-scale shape long before any screen is drawn.
- Architectural design turns that shape into a concrete, layered set of components and services.
- For Web/hypermedia systems specifically, OOHDM then drives content, navigation and interface design from the same conceptual model.
- The result: every screen the user sees is traceable back to a deliberate architectural decision.

Key Takeaways

From high-level structure to the screens users touch

- Architecture is the highest-leverage set of design decisions — choose a style deliberately, based on the genre and dominant quality attribute.
- An Architectural Context Diagram is the fastest way to pin down what a system must exchange with its environment before designing internals.
- Interface, content and navigation design are separable concerns — design content and navigation before locking in the visual interface.
- OOHDM gives Web/hypermedia projects a repeatable path from domain model to a working, navigable, well-organized application.